Digital Design (CS F215) **21-10-2025**

Lab 9: Shift Registers

Lecturer: Harikrishnan N B *Student:*

READ THE FOLLOWING CAREFULLY:

Honour Code for Students: I shall be honest in my efforts and make my parents proud. Write the oath and sign it in your lab notebook.
This lab assignment is a take-home exercise and no in-person evaluations or TA supervision is required.

Marks: 20M

Instructions:

1. **Implementation (Verilog):** Implement in Verilog and simulate with the provided testbenches. Follow the modeling specified in each question, as only solutions using the instructed modeling will be accepted for evaluation.

2. **Submission:** Once you complete the implementation, create a zip file named `CampusID_Lab9.zip` containing all relevant files and upload it to quantaaws.bits-goa.ac.in.

Welcome, developers! Your mission is to build a classic retro-style **Endless Dinosaur Runner Game** from the ground up using Verilog. In this game, the player must jump to avoid oncoming obstacles that move across the screen. You will design each component of the game logic, from player movement to collision detection, and then integrate them into a complete system.

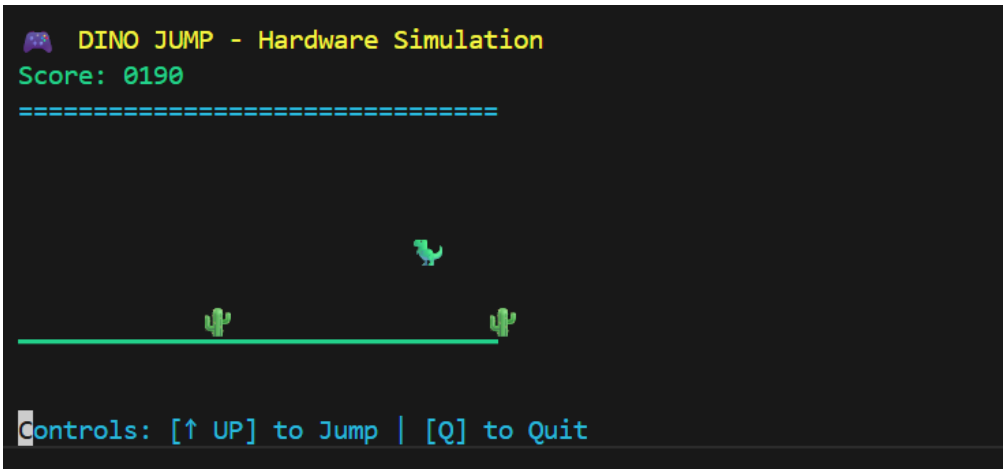

Figure 9.1: The Game

Submission Deadline: **27-10-2025 10:00 PM** **Submission Portal**: quantaaws.bits-goa.ac.in

Implementation Guidelines (Applicable to All Questions)

- You may use `if-else` statements inside an `always @(*)` block if preferred. In such cases, use **internal reg variables** to store intermediate results. However, for purely combinational logic, it is **safer and strongly recommended** to use continuous assign statements to avoid unintentional latches.
- You may declare additional internal `wires` or `reg` signals as needed to organize your logic.
- Do not modify the module ports or parameters under any circumstance—the provided testbench depends on the exact interface and will fail if changed.

Part 1 - Designing the Game Logic Blocks (9M)

Q1: Player Control Logic

Design a combinational module to calculate the player's next vertical position and velocity. Based on the `jump_input` and the previous player state, it must compute the next height and velocity according to simple jump physics. If the game is inactive, the player's motion should freeze.
The module has the following port declaration:

```
module player_control #(
    parameter MAX_JUMP_HEIGHT = 3,
    parameter GROUND_LEVEL = 0
)(
    input  wire jump_input,
    input  wire [2:0] prev_player_height,
    input  wire [2:0] prev_jump_velocity,
    input  wire game_active,
    output wire [2:0] player_height,
    output wire [2:0] jump_velocity
);
```

Your task: Implement the logic inside the module so that it simulates basic jump physics and game state control according to the following conditions:

1. **Jump initiation:** When the jump button (`jump_input`) is pressed **and** the player is on the ground (`prev_player_height == GROUND_LEVEL`) **and** not already jumping (`prev_jump_velocity == 0`), set the new jump velocity to `MAX_JUMP_HEIGHT`.
2. **Jump progression:** If the player is currently jumping (`prev_jump_velocity > 0`), the jump velocity should decrease by 1 each frame until it reaches 0.
3. **Player height update:**
   - If the player is moving upward (`prev_jump_velocity > 0`), increase height by 1.
   - If the player is falling (`prev_jump_velocity == 0` but `prev_player_height > GROUND_LEVEL`), decrease height by 1.

- Otherwise, keep the height constant.

4. **Game active state:** When `game_active` is **low**, both `player_height` and `jump_velocity` should remain **frozen** at their previous values.

Q2: Collision Detector Logic

You are required to implement the `collision_detector` module, which determines whether the player has collided with an obstacle and updates the game-over state accordingly. The module declaration is provided below:

```
module collision_detector #(
    parameter WORLD_WIDTH = 16,
    parameter PLAYER_POSITION = 2,
    parameter GROUND_LEVEL = 0
)(
    input  wire [WORLD_WIDTH-1:0] obstacles,
    input  wire [2:0] player_height,
    input  wire prev_game_over,
    input  wire game_active,
    output wire collision,
    output wire game_over
);
```

Specifications:

1. A collision occurs only when the game is active (`game_active == 1`), an obstacle exists at the player's position (`obstacles[PLAYER_POSITION] == 1`), and the player is on the ground (`player_height == GROUND_LEVEL`).

2. Once a collision occurs, `game_over` should become 1 and remain set (latched) in subsequent frames.

3. When no collision occurs, the module should retain the previous `game_over` state.

Q3: Score Counter Logic

You are required to implement the `score_counter` module that calculates the next score based on obstacle movement and game activity. The module declaration is provided below:

```
module score_counter #(
    parameter WORLD_WIDTH = 16,
    parameter POINTS_PER_OBSTACLE = 10
)(
    input  wire [WORLD_WIDTH-1:0] obstacles,
    input  wire [15:0] prev_score,
    input  wire game_active,
    output wire [15:0] score
);
```

Specifications:

1. The score should increase by `POINTS_PER_OBSTACLE` only when the game is active (`game_active == 1`) and a new obstacle passes the player (represented by `obstacles[0] == 1`).
2. When no new obstacle is detected or the game is inactive, the score remains unchanged.

Part 2 - World and Obstacle Management (6M)

Q4: Universal Shift Register

You are required to implement the `universal_shift_register` module. This is a general-purpose module with four modes of operation, controlled by the `mode` input. The module declaration is provided below:

```
module universal_shift_register #(
    parameter WIDTH = 16
)(
    input  wire [WIDTH-1:0] data_in,
    input  wire [WIDTH-1:0] parallel_in,
    input  wire             serial_in_left,
    input  wire             serial_in_right,
    input  wire [1:0]       mode,
    output wire [WIDTH-1:0] data_out
);
```

Specifications: Implement the logic to perform one of four operations based on the `mode` input:

1. **Mode `2'b00` (Hold):** The output `data_out` should be the same as the input `data_in`.
2. **Mode `2'b01` (Shift Right):** The data should shift right by one bit. The new MSB (`data_out[WIDTH-1]`) should take the value of `serial_in_left`.
3. **Mode `2'b10` (Shift Left):** The data should shift left by one bit. The new LSB (`data_out[0]`) should take the value of `serial_in_right`.
4. **Mode `2'b11` (Parallel Load):** The output `data_out` should be loaded with the value from `parallel_in`.
5. If `data_in` contains invalid values ('x), the output must default to zero (`WIDTH {1'b0}`).

Q5: Obstacle Spawner Logic

You are required to implement the `obstacle_spawner` module, which decides when a new obstacle should appear on the right edge of the game world. The module declaration is provided below:

```
module obstacle_spawner #(
    parameter WORLD_WIDTH = 16,
```

```
    parameter SPAWN_INTERVAL = 8
)(
    input  wire [WORLD_WIDTH-1:0] obstacles_in,
    input  wire [3:0] prev_frame_count,
    input  wire game_active,
    output wire spawn_obstacle,
    output wire [3:0] frame_count
);
```

Specifications:

1. A new obstacle should spawn when the frame counter reaches or exceeds the `SPAWN_INTERVAL` and the two rightmost positions (`obstacles_in[WORLD_WIDTH-1]` and `obstacles_in[WORLD_WIDTH-2]`) are empty (0).
2. The `spawn_obstacle` signal should be 1 only when the above conditions are met and the game is active.
3. When a new obstacle is spawned, the `frame_count` resets to 0. Otherwise, it increments by 1 each frame while the game is active.
4. If the game is inactive, both `spawn_obstacle` and `frame_count` should hold their previous values.

Part 3 - System Integration (5M)

Q6: Top-Level Game Integrator

Integrate all the combinational logic blocks (Q1-Q5) into a single, purely **structural** top-level module, `runner_game_top`. This module will take the entire previous state of the game as inputs (e.g., `prev_obstacles`, `prev_score`) and wire them together to compute and provide the entire next state of the game as its outputs.

Your task:

1. Instantiate each of the five modules you have designed (Q1-Q5).
2. Connect the modules based on the game's data flow. The logic should be as follows: the **Spawner** (Q5) decides if a new obstacle is needed; its output feeds into the **Universal Shift Register** (Q4) to produce the next obstacle state; this new state is then used by the **Collision Detector** (Q2) and **Score Counter** (Q3).
3. The Universal Shift Register's `mode` input should be controlled by the `game_active` signal. When the game is active, it should shift right. When the game is inactive, it should hold its state.
4. You should not write any behavioral logic (e.g., `assign x = a & b;`) in this file, only module instantiations and port connections.

Testing Your Game:

Individual testbenches (`tb_Q1_...v`, `tb_Q2_...v`, etc.) are provided for each of the first five modules. It is highly recommended to test each of your modules in isolation using these dedicated testbenches to verify their functionality before attempting to run the full game. This will make debugging much easier. See README.md for running the testbenches.

Use `python3 game_controller.py` to run the game. Get a score of 100+, take a screenshot, rename it as CampusID_Lab9.png and upload it with rest of the files in a zip file named CampusID_Lab9.zip.

NOTE: The game runs on Unix-based systems (Linux or macOS), and Windows is not supported unless using WSL. Python version 2.x or above can be used, but it must be executed in a Unix environment due to a Unix-specific library. Both `python` and `python3` commands are acceptable.

Marks Distribution:

- `Q1_player_control.v`: 3 Marks
- `Q2_collision_detector.v`: 3 Marks
- `Q3_score_counter.v`: 3 Marks
- `Q4_universal_shift_register.v`: 3 Marks
- `Q5_obstacle_spawner.v`: 3 Marks
- `Q6_runner_game_top.v`: 5 Marks

Total: 20 Marks

END

Debugging Tips

- Check carefully for missing commas/semicolons.
- Check for missing `begin ... end` in `if`, `else` or `case` statements.
- Be mindful of blocking (`=`) vs non-blocking (`<=`) assignments.
- Verify sensitivity lists; `always @ (posedge clk)`, `always @ (*)` etc.